universally agree on the definitions. I chose not to dwell on that topic in this book. I consider unit tests to be isolated tests of a component and integration tests to be tests that exercise multiple components at the same time.

In this section I'm going to show how you can write integration tests for the `StatusMiddleware` from section 36.1 and the API controller from section 36.2. Instead of isolating the components from the surrounding framework and invoking them directly, you'll specifically test them in a context similar to how you use them in practice.

Integration tests are an important part of confirming that your components function correctly, but they don't remove the need for unit tests. Unit tests are excellent for testing small pieces of logic contained in your components and are typically quick to execute. Integration tests are normally significantly slower, as they require much more configuration and may rely on external infrastructure, such as a database.

Consequently, it's normal to have far more unit tests for an app than integration tests. As you saw in chapter 35, unit tests typically verify the behavior of a component, using valid inputs, edge cases, and invalid inputs to ensure that the component behaves correctly in all cases. Once you have an extensive suite of unit tests, you'll likely need only a few integration tests to be confident your application is working correctly.

You could write many types of integration tests for an application. You could test that a service can write to a database correctly, integrate with a third-party service (for sending emails, for example), or handle HTTP requests made to it.

In this section we're going to focus on the last point: verifying that your app can handle requests made to it, as it would if you were accessing the app